EF Core is responsible for translating the LINQ query into SQL to be executed against the database. The SQL generated is dependent on the provider used, taking into account database-specific features.

You can take one of two approaches when writing LINQ queries. You can use *query* syntax or *method* syntax. Query syntax looks similar to SQL statements, and some folk feel more comfortable with it. The following example shows query syntax being used to obtain all cities that belong to a country having a primary key value of `1`:

```
var data = from c in _context.Cities where c.CountryId == 1 select c;
```

However, much like how data annotation attributes only provide a subset of model configuration options, query syntax is not always enough. Some queries can only be expressed using method calls.

My preference is for method syntax, which consists of chaining calls to extension methods on the `IEnumerable` type (http://mng.bz/M0RB). Using query syntax, your code is translated to method calls when it is compiled, so there is no performance difference between the two approaches. We only use method syntax in this book. If you are interested in learning more about query syntax, the official LINQ documentation provides many examples: http://mng.bz/aPNm.

8.3.1 Retrieving multiple records

The first thing you will do is to create a new service for cities that will use the database as a source for their data. Add a new class file to the Services folder, calling this one CityService.cs. This class will implement the `ICityService` interface you created in the last chapter, and it will take